

OrgElMorph

An Onboarding Course in Organic-Electronics Morphology Simulation
Physics · Computation · Differentiation

DiffSim project

Abstract

This course teaches the simulation of *morphology* in organic-electronic thin films — the coupled phase separation and crystallization of polymer/small-molecule/solvent blends that decides how organic solar cells, transistors, and bioelectronic devices perform. It is built around **OrgElMorph**, a GPU-native, differentiable phase-field simulator. Three tracks run in parallel: the *physics* of the models, the *computation* that makes them fast and correct, and the *differentiation* that turns the simulator into an instrument for design and inference. Every concept pairs a written explanation with a runnable tutorial; you reproduce the figures and numbers here, then explore the questions on your own.

Contents

1	Using this course	3
I	Physics	4
2	Binary Cahn–Hilliard: two free energies	5
2.1	The model	5
2.2	Two bulk free energies	6
2.3	Run it	6
2.4	Code walk-through	8
2.5	Explore on your own	9
II	Computation	10
3	Convergence: basis order and time integration	11
4	Boundary conditions, numerically	12
5	Spatial and temporal adaptivity	13
6	The solver ecosystem	14
7	Two dimensions versus three	15
III	Differentiation	16
8	What differentiable simulation is	17
9	Morphology sensitivity	18
10	Recovering parameters from a morphology	19
11	Process gradients	20
12	Crystallinity sensitivity	21
13	Learning the free energy from snapshots	22

Chapter 1

Using this course

Read a concept below, then open its tutorial folder (`tutorials/orgelmorph-course/<track>/<NN_name>/`), run `run.py`, and compare your printed self-check table and figures with the ones in this document. The tutorials call the same production solver the research code uses. The three tracks cross-reference each other; a first pass straight down the Physics track is a good start, dipping into the matching Computational and Differentiable concepts as they are referenced.

Boxes flag the two things you act on:

Key idea. The load-bearing concept.

Run it. The command, and what to compare.

Questions marked **Q** are for you to explore — they have no answers printed here on purpose.

Part I

Physics

Chapter 2

Binary Cahn–Hilliard: two free energies

Key idea. A blend of two components lowers its free energy by *demixing*. The Cahn–Hilliard equation is the conserved gradient flow of a Ginzburg–Landau free energy; that free energy can only decrease in time, and it splits into a *bulk* (mixing) part and an *interfacial* (gradient) part whose competition sets the morphology and its length scale.

2.1 The model

Let $c(\mathbf{x}, t)$ be the local volume fraction of one component (a *conserved* order parameter: the total amount of each species is fixed). The Ginzburg–Landau free energy is

$$F[c] = \int_{\Omega} \left[f(c) + \frac{\kappa}{2} |\nabla c|^2 \right] dV, \quad (2.1)$$

with two contributions:

- the **bulk** (homogeneous) free energy density $f(c)$, which is minimized by the pure phases and *penalized* in between — this is what drives demixing;
- the **interfacial** term $\frac{\kappa}{2} |\nabla c|^2$, which penalizes spatial variation of c and therefore *opposes* demixing at short scales. The coefficient $\kappa > 0$ has units of energy $\times$ length² and sets the interface width $\ell \sim \sqrt{\kappa}$.

Because c is conserved, the dynamics is the *conserved* (Model-B) gradient flow of (2.1): mass moves down gradients of the chemical potential $\mu = \delta F / \delta c$,

$$\frac{\partial c}{\partial t} = \nabla \cdot (M \nabla \mu), \quad \mu = f'(c) - \kappa \nabla^2 c. \quad (2.2)$$

Here $M > 0$ is the mobility (how fast mass responds to a chemical-potential gradient; it sets the *time* scale but not the equilibrium morphology). The fourth-order PDE is split into the two second-order equations (2.2) — the *mixed* (c, μ) form the solver uses.

F is a Lyapunov functional. Taking dF/dt along (2.2) and integrating by parts with no-flux boundaries gives $dF/dt = -\int_{\Omega} M |\nabla \mu|^2 dV \leq 0$. The free energy can only decrease.

Watching it decrease — and split between bulk and interface — is how you *see* the physics in the numbers.

2.2 Two bulk free energies

Polynomial double well (the canonical model). The simplest f with two equal minima,

$$f_{\text{poly}}(c) = \frac{1}{4}(c^2 - 1)^2, \quad f' = c^3 - c, \quad f'' = 3c^2 - 1, \quad c \in [-1, 1]. \quad (2.3)$$

The two phases are $c = \pm 1$; the region $f'' < 0$ (i.e. $|c| < 1/\sqrt{3}$) is *spinodally unstable* — any small fluctuation there grows. This model is dimensionless and symmetric; it isolates the phase-field mechanism from material detail.

Flory–Huggins (the physical model). For a real blend the mixing free energy is entropic + enthalpic,

$$f_{\text{fh}}(c) = A [c \ln c + (1 - c) \ln(1 - c)] + B c(1 - c), \quad c \in (0, 1), \quad (2.4)$$

where the logarithmic *entropy of mixing* (scale $A \sim 1/N$, with N the degree of polymerization) always favors mixing, and the *enthalpic* term $B c(1 - c)$ (with B the Flory interaction parameter χ) favors demixing when the components dislike each other. The curvature

$$f''_{\text{fh}}(c) = A \left(\frac{1}{c} + \frac{1}{1-c} \right) - 2B \quad (2.5)$$

is negative — unstable — only where the enthalpic $-2B$ beats the (always convex) entropic term. This is the *spinodal*. The logs diverge at the pure phases, so Flory–Huggins keeps c strictly inside $(0, 1)$ (no unphysical overshoot), at the cost of a stiff f'' near the walls — a numerical point we return to in the Computational track.

Coefficients and their ranges. $M > 0$ (mobility, time scale). $\kappa > 0$ (gradient penalty; interface width $\sqrt{\kappa}$ — choose it so the interface spans ~ 3 – 5 mesh cells). Polynomial: no free parameters. Flory–Huggins: $A > 0$ (entropy, $\sim 1/N$; $A \rightarrow 0$ for long polymers), $B = \chi$ (interaction; demixing requires B large enough that $f'' < 0$ somewhere — for $A = 1$ the symmetric point $c = \frac{1}{2}$ is unstable once $B > 2A = 2$). The tutorial uses $A = 1$, $B = 2.5$ (a clear quench); real blends map $A \sim 1/N$, $B \sim \chi$ (see `materials/materials.yaml`).

2.3 Run it

Run it.

```
cd physics/01_ch_binary_energies
python run.py                # both free energies, 128x128
```

It marches a spinodal decomposition from a nearly uniform blend ($c = \bar{c}$ + small noise) for both free energies, printing the energy budget. Compare the printed table with `EXPECTED.md` and the figures below.

What you should see. From a nearly uniform, slightly noisy blend, small fluctuations in the unstable region grow (spinodal decomposition), sharpen into domains separated by thin interfaces, and then *coarsen*. The total free energy drops monotonically (after a one-step start-up transient discussed below), mass is conserved to solver tolerance, and the field saturates at the two coexisting phases.

On the random seed. The initial perturbation is drawn from a *fixed* random seed (set in `spinodal.py`), so every run is bit-reproducible and matches the figures here — that is what lets you *compare* against the document. Note the distinction: the *statistics* (energy decay, mass conservation, the phase values $c_{\min}, c_{\max}$) are seed-independent physics, but the *exact morphology* — which domain forms where — is not. Change the seed and the pattern changes while the energy curve does not. (Later, in the Computational and Physics-noise concepts, the seed becomes a first-class object: convergence must be seed-robust, and thermal *nucleation* is seeded by physical fluctuations.)

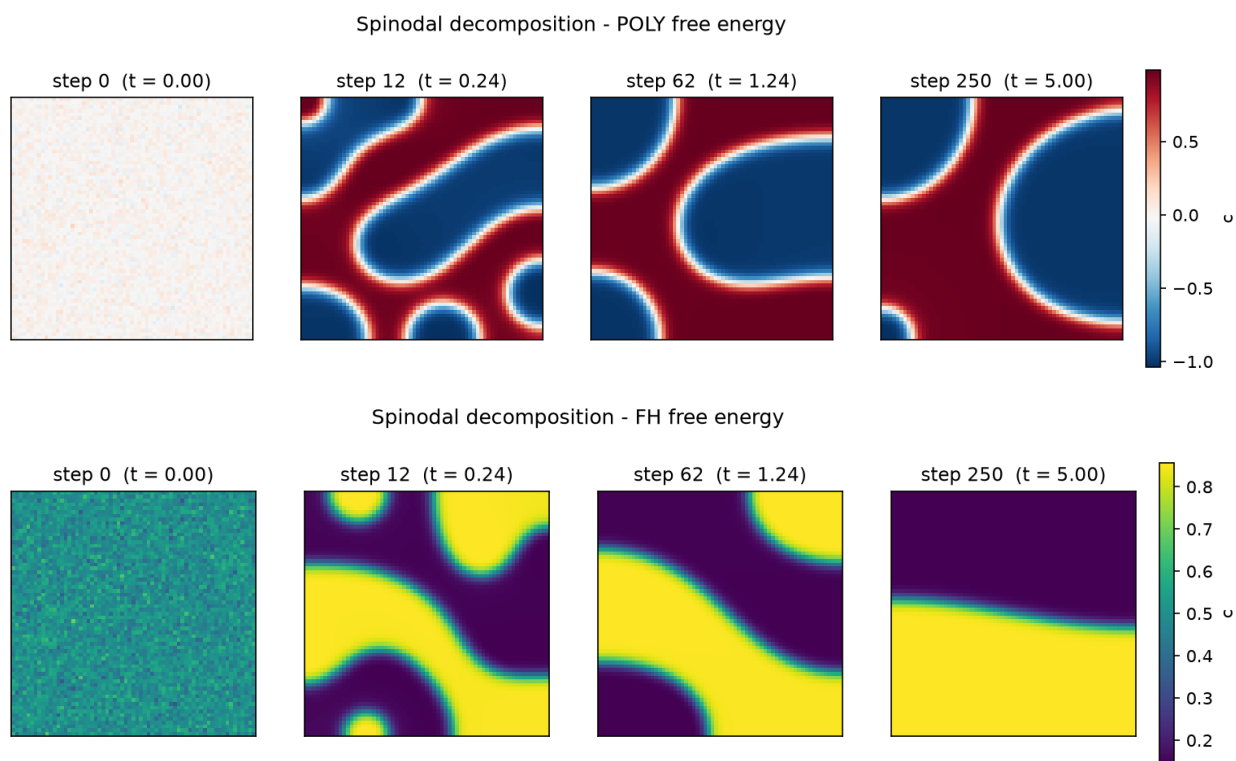


Figure 2.1: Spinodal decomposition and coarsening for the polynomial (top, $c \in [-1, 1]$) and Flory–Huggins (bottom, $c \in (0, 1)$) free energies, at the mesh and parameters of the tutorial. Same mixing physics, different order-parameter range and interface structure.

Measured self-check (your `run.py` should reproduce these to a few significant figures; exact values depend on the RNG seed fixed in the tutorial).

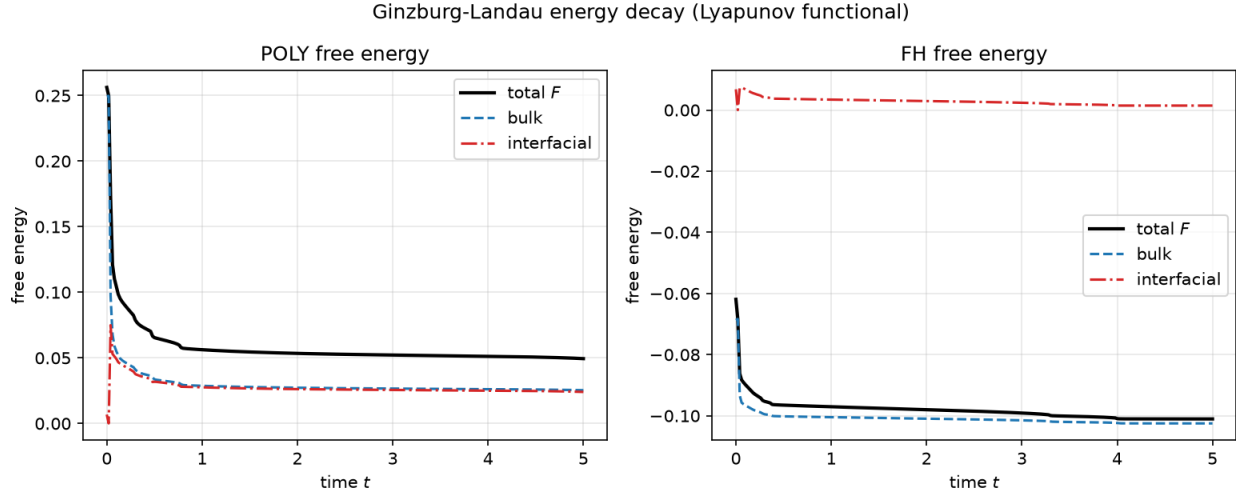


Figure 2.2: The Ginzburg–Landau energy budget: total free energy (black) and its bulk (blue) and interfacial (red) parts versus time. The total is a Lyapunov functional — monotone decreasing. Early on, forming interfaces *raises* the interfacial energy while the bulk energy falls faster; during coarsening, domains merge and the interfacial energy falls as total interface area shrinks.

	polynomial	Flory–Huggins
total F : start $\rightarrow$ end	0.256 $\rightarrow$ 0.0494	-0.0619 $\rightarrow$ -0.101
interfacial energy: peak $\rightarrow$ end	0.0749 $\rightarrow$ 0.024	0.00741 $\rightarrow$ 0.00146
phase values $[c_{\min}, c_{\max}]$	$[-1.04, 0.99]$	$[0.14, 0.86]$
mass drift $ \Delta m $	4.1×10^{-16}	$< 10^{-14}$ (machine)
F monotone after step 1	yes	yes

2.4 Code walk-through

The tutorial’s physics lives in `spinodal.py`, which drives the production brick `src/diffsim/physics/cahn_hilliard.py`. The three sections you interact with:

1. Build a mesh and the stepper.

```
dm, mesh, cons = build_mesh_dm(level=7, p=1, device="cuda:0") # 128x128
st = CahnHilliardStepper(dm, M, kappa, dt, order=1,
                        energy="poly") # or energy="fh"
```

energy selects $f(c)$; order is the time scheme (1 = backward Euler); M , $kappa$ are the coefficients above. The constructor is where the *model* is chosen — everything else is running it.

2. Seed the initial blend.

```
st.set_initial(lambda x: c_avg + amp*rng.standard_normal(len(x)),
              mu_init="consistent")
```

A nearly uniform field with a small random perturbation — the seed of the spinodal instability. `mu_init="consistent"` projects a matching initial chemical potential so the very first step does not see an artificial μ -transient (the one-step start-up you see in Fig. 2.2).

3. March, and measure the energy each step.

```
for n in range(steps):  
    c, mu = st.step()  
    F_total, F_bulk, F_int = energy_budget(st, dm, mesh, c, ...)
```

`step()` advances one time step (assemble $\rightarrow$ Newton solve on the GPU). `energy_budget` evaluates c and ∇c at the Gauss points — using the same quadrature the solver uses — and integrates the two energy densities of (2.1) separately. That is the whole measurement: no post-processing magic, just the definition of F .

2.5 Explore on your own

Q2.5.1 Vary κ over, say, $\{2, 5, 10\} \times 10^{-4}$. How does the equilibrium interface width scale, and how many mesh cells does the interface span in each case? At what point do you stop trusting the result, and why?

Q2.5.2 For Flory–Huggins, sweep B (the interaction χ) from below to above the spinodal threshold $2A$. Where does demixing stop happening, and does the onset match $f''(\frac{1}{2}) = 0$?

Q2.5.3 The polynomial model is symmetric ($\bar{c} = 0$); Flory–Huggins with $\bar{c} \neq \frac{1}{2}$ is not. Run an off-critical blend ($\bar{c} = 0.35$) and describe how the morphology differs from the symmetric case (droplets vs. bicontinuous). Which blend ratio gives which?

Q2.5.4 The total energy dips monotonically but the interfacial energy is *non-monotone*. Explain the two regimes (interface formation vs. coarsening) in terms of F_{bulk} and F_{int} , and predict how each scales during coarsening.

Q2.5.5 Mobility M was said to set the time scale but not the equilibrium. Test it: double M and check whether the *final* morphology and energy are unchanged while the *approach* rescales in time.

Part II

Computation

Chapter 3

Convergence: basis order and time integration

Key idea. A discretization is only trustworthy if it *converges* at the rate the theory predicts. The method of manufactured solutions (MMS) measures that rate directly: plug a known solution into the equations, add the residual as a source, and check that the error falls like h^{p+1} in space and Δt^q in time.

This concept walks through the Cahn–Hilliard convergence gates that ship with the solver: $p = 1$ vs $p = 2$ spatial order, and BDF1 vs BDF2 temporal order, each measured against a manufactured solution. It connects to Physics Chapter 2 (the same brick) and to the adaptive-stepping concept (why variable-coefficient BDF2 matters once Δt changes).

Under construction — follows the Physics 2 template (equations, run-it, measured self-check, code walk-through, questions).

Chapter 4

Boundary conditions, numerically

Under construction.

Chapter 5

Spatial and temporal adaptivity

Under construction — octree refinement and the LTE/PI step controller.

Chapter 6

The solver ecosystem

Under construction — direct (cuDSS) vs the `blockch` block preconditioner vs matrix-free, and when each wins (with the measured tables).

Chapter 7

Two dimensions versus three

Under construction — cost, memory, and the assembly/solve story at scale.

Part III

Differentiation

Chapter 8

What differentiable simulation is

Key idea. A classical solver answers “*what happens?*”. A differentiable solver also answers “*what should I change?*” — it returns the gradient of any output with respect to any input (material parameters, process schedule, geometry), so the simulator becomes an instrument for design and inference, not just prediction.

This concept introduces the three ways DiffSim computes a gradient — a hand-derived adjoint, an autograd twin, and finite differences — and the three-way check that keeps them honest. *Under construction.*

Chapter 9

Morphology sensitivity

Under construction — gradient of a morphology observable (domain size, interfacial area) with respect to the interaction parameter χ .

Chapter 10

Recovering parameters from a morphology

Under construction — fit χ , N to a target morphology (the Wodo-2012 “learn the free-energy functional” story, at tutorial scale).

Chapter 11

Process gradients

Under construction — gradient with respect to evaporation rate and quench schedule.

Chapter 12

Crystallinity sensitivity

Under construction — differentiating through the ψ, θ crystallization fields.

Chapter 13

Learning the free energy from snapshots

Under construction — recover the free-energy functional from a time series of morphology snapshots.

Chapter 14

Capstone: inverse-designing a process

Under construction — optimize an evaporation schedule to hit a target domain size or crystallinity.